

TOOL-CALLING PROTOCOLS ARE DIRECTED CONTAINERS

MACBETH

ABSTRACT. An agent tool-calling protocol – the discipline by which a language-model agent issues a typed request and receives a typed response – is a container: its shapes are the request schemas and its positions are the admissible responses. We observe that the *harness* running such a protocol is not a separate meta-object but the same container realized as a comonad, so a protocol is a directed container, hence a small category, hence a polynomial comonad; the cofree comonad on the tool functor is the interaction trace, and stateful conversational context is a store-comonad worker. For a single agent this store structure is exactly curried state-threading and buys nothing. Its value appears only under composition: interleaving two interacting stateful protocols is a comonad distributive law, equivalently a Zappa–Szép weld of the underlying directed containers, which does not exist for free. Once a freeness condition on the two protocols holds, whether a consistent interleaving exists is governed by a single cohomology class $[\omega] \in H^2$, the weld existing exactly when $[\omega] = 0$; at the smallest re-entrant model the class is one bit $[\omega] = \varepsilon \in \mathbb{Z}/2$, machine-checked in Lean 4. The note claims no new theorem; its contribution is the identification, and a positioning against the coalgebraic and effect-handler semantics of protocols that stop one refinement short of this structure.

Status. Exploratory work-in-progress note (Kodamai containers series); corresponds to `scotmacbeth/work-in-progress` content-commit `6a66fe3`. Every mathematical fact invoked is proved, and where stated machine-checked, elsewhere; the contribution here is the synthesis and its honest positioning, not a new theorem. The single result specific to this note’s running model – the value of the obstruction class at the smallest re-entrant protocol – is verified in Lean 4 (§5, Remark 3). Not yet reviewed.

1. INTRODUCTION

An agent built on a language model does its work by *calling tools*. It emits a structured request – a function name with typed arguments, a database query, a call to another agent – and the surrounding runtime, the *harness*, executes that request and returns a structured response, which the agent folds back into its context before deciding what to do next. The governing discipline – which requests are well-formed, which responses each request may receive, how the responses accumulate into a growing context – is what recent engineering practice, following the Model Context Protocol and its relatives, calls a *tool-calling protocol*. Multi-agent systems compose these: a supervisor agent calls a worker agent as a tool, two agents read and write a shared store, an agent calls back into the very supervisor that invoked it.

The engineering of such systems is well developed; their *semantics* is not. What kind of mathematical object is a tool-calling protocol? What object is the harness that runs it? And when two agents interact, what decides whether their composite behaves consistently, or instead diverges into the re-entrancy and stale-read defects that plague concurrent stateful systems? These questions are usually answered operationally, one system at a time, and the correctness of a multi-agent interleaving is typically discovered the way concurrency bugs always are: at run time, after it breaks.

The gap. There is a natural home for these questions, and it has been standing empty. A container [3] is a set of shapes S together with a family of positions $P : S \rightarrow \mathbf{Set}$; its extension is the polynomial functor $\llbracket S \triangleright P \rrbracket(X) = \sum_{s:S} (P s \rightarrow X)$. Reading S as the request schemas and $P s$ as the admissible responses to schema s , a single protocol step *is* a container. This much is not new: a session coalgebra in the sense of Keizer, Basold and Pérez [4] is a coalgebra

$c : X \rightarrow F(X)$ for exactly this endofunctor $F(X) = \coprod_{a \in A} X^{B_a}$, and they trace the observation to Gambino and Kock. What has not been done is to carry the identification past the single step. A session coalgebra is a coalgebra for the protocol’s *transition* map; it is not a *comonad* on a protocol object, it carries no cofree interaction trace, no substitution or tensor algebra of protocols, no notion of stateful context threaded across steps, and no account of when two protocols compose. The one published line that treats tool-calling algebraically – Segura’s Lawvere-theory handlers for prompt pipelines [7] – is protocol-agnostic and does not model the protocol object at all. The comonadic, directed-container layer – precisely the layer that the theory of directed containers exists to supply – is unclaimed.

The observation. The missing ingredient is the harness. A tool-calling protocol is not run by a meta-level machine that stands outside it; the harness is the discipline of threading context from one step to the next, and that discipline is *structure on the same container*. A container whose extension carries a comonad structure – a counit “answer from the current context” and a comultiplication “the context of the context” – is exactly a *directed container* in the sense of Ahman and Uustalu [1], and directed containers are, up to equivalence, small categories, and up to equivalence polynomial comonads. So the harness upgrades the protocol from a bare container to a directed container, and the seed equivalence

$$(1) \quad \text{DCont} \simeq \text{Cat} \simeq \text{Cat}^\# \subset \text{Poly}$$

(directed container $\simeq$ small category $\simeq$ polynomial comonad) does the rest of the lifting for free. Once the protocol is a comonad, three further structures are immediate rather than invented: the *cofree comonad* on the tool functor is the interaction trace, the tree of all runs of the protocol; the store comonad ΔS models conversational *state*; and the substitution $\triangleleft$ and Dirichlet tensor $\otimes$ of [6, 5] are the two ways of composing protocols in sequence and in parallel.

The main point, stated as a dichotomy. The state structure is worth its cost only under composition, and the sharp statement is a dichotomy we import rather than prove. For a single agent, the store worker is exactly hand-threaded state (Proposition 2): a getter and a setter, no more. For two *interacting* stateful agents – ones that read and write each other’s state – the composite is neither the free parallel tensor nor a bare nesting but a *Zappa–Szép weld*, and a weld is a distributive law saying how the two contexts interleave. Comonads, unlike the data they carry, do not compose for free [2]. Whether a consistent interleaving exists is a single cohomology class.

Theorem 1 (imported; §5). *Let two interacting stateful protocols be presented as directed containers C, D . A consistent interleaving of their harnesses – a comonad distributive law, equivalently a Zappa–Szép weld $C \bowtie D$ – exists if and only if a freeness condition (L) holds and an obstruction class*

$$[\omega] \in H^2 \quad \text{vanishes,} \quad (G) \iff [\omega] = 0.$$

At the smallest re-entrant supervisor–worker model the class lives in $H^2 \cong \mathbb{Z}/2$ and equals a single bit,

$$[\omega] = \varepsilon \in \mathbb{Z}/2, \quad \varepsilon = \text{“does the callee mutate state the caller depends on?”},$$

so the weld exists iff $\varepsilon = 0$. This value is machine-checked in Lean 4.

The engineering reading is the payoff. A wrong interleaving – a distributive law that violates the coherence axioms – is not an abstract nuisance; it is precisely the state-divergence and re-entrancy defect, where one agent reads context that another has already invalidated. Hand-threading the shared state does not escape this: every call site that reads one register and writes another silently commits to an order of effects, an implicit distributive law, and nothing checks that the local choices are globally consistent. A program can hand-thread its way into a globally inconsistent interleaving one locally reasonable line at a time, and that inconsistency is $[\omega] \neq 0$, found at run time as a bug. The comonadic view does not make the composite exist – no formalism can, because sometimes it genuinely does not – but it *concentrates the*

decision: the interleaving is one named object, its soundness is one class, and both can be inspected before the system runs.

Contribution and honest scope. This note proves no new theorem. Its contribution is the identification of the tool-calling protocol and its harness with the directed-container / polynomial-comonad structure, the resulting reading of the trace, state, and composition operators, and an honest positioning against the two nearest precedents [4, 7]. Every mathematical fact it uses is established elsewhere: the equivalence chain (1) is Ahman–Uustalu [1]; the distributive-law $\Leftrightarrow$ Zappa–Szép correspondence and the $[\omega] \in H^2$ obstruction are from the directed-container and bicrossed-product literature and our own prior work; the value $[\omega] = \varepsilon$ at the smallest model is machine-checked (Remark 3). We draw the formalisation boundary explicitly where it falls (Remark 3), and we flag the one reduction that is invoked at the directed-container level and not yet restated directly for store comonads as an open problem (§7).

Outline. Section 2 recalls containers, the equivalence chain, the cofree and store comonads, and the obstruction we import. Section 3 identifies the protocol with a container and the harness with its comonad, and reads off the interaction trace. Section 4 gives the single-agent verdict: state is threaded state, and buys nothing. Section 5 gives the composition verdict: interleaving is a distributive law and its soundness is $[\omega] = 0$, with the Lean boundary. Section 6 positions the note against session coalgebras, effect handlers, and process-calculus accounts. Section 7 states the open problems.

2. PRELIMINARIES

We recall only what the argument uses; a reader fluent in containers and Poly can skim to §3. Throughout, **Set** is the base and y denotes the identity container $1 \triangleright (\lambda_. 1)$, whose extension is the identity functor.

Definition 1 (Container, extension, morphism [3]). A *container* $S \triangleright P$ consists of a set of *shapes* S and a family of *positions* $P : S \rightarrow \mathbf{Set}$. Its *extension* is the polynomial functor

$$\llbracket S \triangleright P \rrbracket(X) = \sum_{s:S} (P s \rightarrow X).$$

A *morphism* $(S \triangleright P) \rightarrow (T \triangleright Q)$ is a pair $(f, f^\#)$ of a forward map $f : S \rightarrow T$ on shapes and a backward, contravariant family $f^\# : \prod_{s:S} (Q(f s) \rightarrow P s)$ on positions. Containers and their morphisms form the category **Cont**; extension is a full and faithful embedding $\mathbf{Cont} \hookrightarrow [\mathbf{Set}, \mathbf{Set}]$ onto the polynomial functors.

Definition 2 (Directed container; the equivalence chain [1]). A *directed container* is a container $S \triangleright P$ together with operations making each position set the arrows out of a shape: a unit $o : \prod_s P s$, a “sub-shape” operation $s \oplus - : \prod_s (P s \rightarrow S)$, and a composition of positions, subject to category-like axioms. The resulting data is exactly a small category: shapes are objects, $P s = \coprod_t \mathbf{Cat}(s, t)$ collects the arrows out of s . Passing to extensions, a directed container is precisely a *polynomial comonad* $(\llbracket S \triangleright P \rrbracket, \epsilon, \delta)$, with counit $\epsilon : \llbracket S \triangleright P \rrbracket \Rightarrow \mathbf{Id}$ and comultiplication $\delta : \llbracket S \triangleright P \rrbracket \Rightarrow \llbracket S \triangleright P \rrbracket \triangleleft \llbracket S \triangleright P \rrbracket$. This gives the equivalence chain (1), $\mathbf{DCont} \simeq \mathbf{Cat} \simeq \mathbf{Cat}^\#$.

Definition 3 (Substitution and tensor [6, 5]). For containers $C = S \triangleright P$ and $D = T \triangleright Q$, *substitution* $C \triangleleft D$ has extension $\llbracket C \rrbracket \circ \llbracket D \rrbracket$ (nest a D -structure at each position of a C -structure); it is the composition of the corresponding polynomial functors. The *Dirichlet tensor* $C \otimes D$ multiplies shapes and positions, $(S \times T) \triangleright (\lambda(s, t). P s \times Q t)$; on extensions it is the Day convolution for the pointwise product. Substitution models sequential composition; the tensor models independent parallel composition.

Definition 4 (Cofree comonad). For a container F with extension $\llbracket F \rrbracket$, the *cofree comonad* $\mathbf{Cofree} F$ is the terminal F -coalgebra structure carried by the greatest fixed point $\nu X. (A \times \llbracket F \rrbracket X)$ relative to a label object A ; concretely $\mathbf{Cofree} F(A)$ is the set of A -labelled trees branching by the positions of F . It is the universal comonad mapping to F : every run of the one-step functor F extends uniquely to an element of the trace $\mathbf{Cofree} F$.

Definition 5 (Store comonad and state object [5]). For a set S , the *state object* is the container $\Delta S := S \triangleright (\lambda_- . S)$, with extension

$$\llbracket \Delta S \rrbracket(X) = \sum_{s:S} X^S = S \times X^S = \text{Store}_S X,$$

the *store* (costate) comonad: a current state paired with a reader $X^S = (S \rightarrow X)$. It is a directed container – the codiscrete category on S , in which every pair of objects has a unique arrow. Its counit reads the current state, and its comultiplication duplicates the register. A *worker of state S from X to Y* is a container morphism $w : \Delta S \otimes X \rightarrow Y$.

Unfolding Definition 1 for a worker $w : \Delta S \otimes X \rightarrow Y$ with $X = X_0 \triangleright X_1$ and $Y = Y_0 \triangleright Y_1$ gives a forward map and a backward family

$$(2) \quad \text{fwd} : S \times X_0 \rightarrow Y_0, \quad \text{bwd} : \prod_{(s, x_0)} (Y_1(\text{fwd}(s, x_0)) \rightarrow S \times X_1 x_0),$$

and, by the universal property of the product, **bwd** splits into a *state-writeback* $\text{bwd}_S : \prod Y_1(\cdots) \rightarrow S$ and an ordinary back-map $\text{bwd}_X : \prod Y_1(\cdots) \rightarrow X_1 x_0$. The forward leg reads state and input and answers; the backward leg writes back a new state and answers the input. This is the whole of “a stateful step,” typed.

Definition 6 (Zappa–Szép weld; the obstruction we import [2]). A *Zappa–Szép weld* $C \bowtie D$ of two directed containers is a directed-container structure on a shared underlying object in which each of C and D acts on the other, restricting to C and D on the two factors. Welds correspond to distributive laws $\lambda : \llbracket D \rrbracket \triangleleft \llbracket C \rrbracket \Rightarrow \llbracket C \rrbracket \triangleleft \llbracket D \rrbracket$ (equivalently the comonad-distributive dual) satisfying the coherence axioms; at the level of the underlying categories this is the classical bicrossed product. A weld exists iff a freeness condition (L) holds and a global-closure condition (G) holds, and the obstruction to (G) is a single class $[\omega] \in H^2$ with $(G) \Leftrightarrow [\omega] = 0$. We invoke these facts; they are proved in the cited work and our own.

3. A PROTOCOL IS A CONTAINER; THE HARNESS IS ITS COMONAD

We fix terminology at the level of generality the note needs: a tool-calling protocol is any discipline of typed requests and typed responses, of which the Model Context Protocol is one concrete instance. We do not model any particular wire format; we model the shape of the interaction.

Definition 7 (Tool-calling protocol). A *tool-calling protocol* is a container $S \triangleright P$ in which S is the set of well-formed *request schemas* an agent may emit (a tool name together with typed arguments, a query, a delegated sub-task) and, for each $s : S$, $P s$ is the set of admissible *responses* the harness may return to s . One *step* of the protocol is the endofunctor $\llbracket S \triangleright P \rrbracket(X) = \sum_{s:S} (P s \rightarrow X)$: choose a request, and continue as a function of whichever response comes back.

Remark 1 (What is conceded). Definition 7, up to the reading of S and P , is not new. The step endofunctor is the session-coalgebra functor $F(X) = \coprod_{a \in A} X^{B_a}$ of Keizer, Basold and Pérez [4], who study coalgebras $c : X \rightarrow F(X)$ for it (they cite Gambino–Kock, and do not use the word “container”). We take “one protocol step is a container” from them. Everything from here on – the comonad, the trace, the store worker, the composition operators, and the obstruction – lies outside their development; see §6.

The step functor alone is only half the object. An agent does not issue one request in a vacuum; the harness *threads context* – the growing transcript of prior requests and responses – from each step to the next, and it is this threading, not the individual step, that makes a protocol a protocol. The claim of this section is that the threading is not an external machine but a comonad structure on the very same container.

Proposition 1 (The harness is the protocol’s comonad). *A tool-calling protocol whose harness threads context coherently – an initial (empty) context, a way to read the current context to answer without a further request, and an associative extension of a context by a step – is a directed container: the counit ϵ is “answer from the current context,” the comultiplication δ is*

“expose the context of each continuation,” and the category-theoretic axioms of Definition 2 are exactly the coherence of context threading. By the equivalence chain (1) it is therefore a small category and a polynomial comonad.

Proof. This is the content of Definition 2 read in the protocol’s vocabulary, not a new theorem. A coherent threading discipline provides precisely the unit o (the empty context is a distinguished position at each shape), the sub-shape map $s \oplus -$ (a response determines the continuation’s schema), and the composition of positions (concatenation of transcripts), subject to the directed-container axioms, which say that the empty context is a two-sided unit for concatenation and that concatenation is associative. These are the comonad laws for $(\llbracket S \triangleright P \rrbracket, \epsilon, \delta)$ under (1). $\square$

The harness is thus not a meta-object above the protocol; it *is* the protocol, with the comonad structure that the word “harness” names. Two consequences are immediate.

Corollary 1 (The interaction trace is the cofree comonad). *The interaction trace of a protocol $S \triangleright P$ – the tree of all possible runs, each run a sequence of request/response pairs – is the cofree comonad $\text{Cofree } S \triangleright P$ of Definition 4 on the tool functor, with labels drawn from the agent’s observable outputs. A single conversation is one root-to-node branch; the comonad’s counit reads the current node, and its comultiplication re-roots the trace at each node, giving the sub-conversation visible from there.*

Remark 2 (Running the agent is an interaction law). The comonad reading also locates the agent. An agent’s tool-issuing behaviour is a monad of effects, and executing it against the harness is the pairing of that monad with the protocol comonad by an *interaction law* in the sense of Katsumata, Rivas and Uustalu [8]: the categorical form of “running” the agent against the protocol to produce a value. We do not develop the agent-monad here – this note is about the protocol object – but record that the harness-comonad identification makes “run” a standard construction rather than an operational primitive.

Corollary 2 (Protocols compose in two ways). *Two protocols compose by substitution $C \triangleleft D$ – run D at each response position of C , i.e. a tool whose responses are themselves protocols (a supervisor delegating to a sub-protocol) – and by Dirichlet tensor $C \otimes D$ – offer both protocols’ requests independently (two non-interacting tools). These are Definition 3. The interesting composite, of two protocols that share state, is neither, and is the subject of §5.*

4. THE SINGLE-AGENT VERDICT: STATE IS THREADED STATE

Conversational context that the agent may also *mutate* – a scratchpad, a running plan, a budget – is modelled by giving the worker a state object ΔS (Definition 5). Before claiming anything for this, we grant the sceptic the single-agent case in full.

Proposition 2 (One worker is threaded state, re-spelled). *Fix a state set S . The elements of the store $\text{Store}_S X = S \times X^S$ are in natural bijection with state-threading data: a current state together with a map from states to outputs. On the lens fragment $X = y$, $Y = A \triangleright (\lambda_.B)$, a worker $\Delta S \rightarrow A \triangleright (\lambda_.B)$ is exactly a pair*

$$\text{get} : S \rightarrow A, \quad \text{put} : S \times B \rightarrow S,$$

the ordinary “read the state to answer; write the state back” of hand-threaded code.

Proof. The bijection $\text{Store}_S X = S \times X^S$ is the definition of $\llbracket \Delta S \rrbracket$. Specialising (2) to $X = y$ (so $X_0 = X_1 = 1$) and $Y = A \triangleright (\lambda_.B)$ gives $\text{fwd} : S \rightarrow A$, i.e. get , and $\text{bwd}_S : \prod_s (B \rightarrow S)$, i.e. $\text{put} : S \times B \rightarrow S$, with bwd_X trivial. $\square$

There is no daylight here. A store is a state and a reader; a lens is a getter and a setter; threading a state through a signature gives a getter and a setter. The comonad’s counit is “read the current state” (get) and its comultiplication is “duplicate the register” – operations hand-threaded code already performs. For a system of one stateful agent the store comonad is a rename: more category theory, no more safety. Honesty requires saying so plainly, and

it is the reason the value must be sought elsewhere. It is not sought in vain, because agentic systems are not systems of one agent.

5. THE COMPOSITION VERDICT: INTERLEAVING IS A DISTRIBUTIVE LAW

Everything the state structure is worth lives in how two stateful agents combine, and here the store comonad stops being a rename, because comonads – unlike the data they carry – do not compose for free [2].

Two ways to stack, only one free. Tensoring two state objects multiplies their states on the nose,

$$(3) \quad \Delta S \otimes \Delta T = \Delta(S \times T),$$

whereas substitution nests a store inside a store,

$$(4) \quad \Delta S \triangleleft \Delta T = (S \times T^S) \triangleright (\cdots) \neq \Delta(S \times T),$$

recording not a pair of states but a whole branch map $S \rightarrow T$; at $|S| = |T| = 2$ these are $4 \cdot y^4$ against $8 \cdot y^4$, an explicit retract, not an equality. Equation (3) is the composite of two agents that do *not* interact – independent registers, no shared reads or writes – and it is forced, “ask both, answer both.” It is free precisely because it assumes non-interference.

The interacting composite is neither. The agents an orchestration actually composes do interact: a supervisor over a worker that can call back into the supervisor (re-entrancy), or two agents that read and write a shared ledger, each acting on data the other produced. There the registers are not independent, so the composite is not $\Delta S \otimes \Delta T$; and it is not a bare nesting either. It is a Zappa–Szép weld $\Delta S \bowtie \Delta T$ (Definition 6), in which each store acts on the other, and a weld is a distributive law λ recording how the two contexts interleave: which agent reads first, whose write the other sees. Comonads do not compose for free – without a valid λ there is no composite protocol at all, only two protocols and an unanswered question about the order of their effects.

Existence is a cohomology class. By Theorem 1, once (L) holds the weld exists exactly when $[\omega] = 0 \in H^2$. At the smallest re-entrant supervisor–worker model the class is one bit

$$(5) \quad [\omega] = \varepsilon \in \mathbb{Z}/2, \quad \varepsilon = \text{“does the worker mutate shared state the supervisor depends on?”},$$

so the weld exists iff $\varepsilon = 0$. A *wrong* interleaving – a λ violating the coherence axioms – is not an abstract nuisance: it is the state-divergence / re-entrancy defect, the concrete failure where one agent reads context another has already invalidated.

Remark 3 (Where the machine check stops). The class computation (5) is machine-checked in Lean 4, `sorry-free` and `Mathlib-free` (`Reentrancy.lean`): the file certifies $\varphi(\omega \varepsilon) = \varepsilon$ and $\text{InB2}(\omega \varepsilon) \Leftrightarrow \varepsilon = \text{false}$, i.e. that the cocycle $\omega(\varepsilon)$ is a coboundary exactly when $\varepsilon = 0$. What Lean certifies is the finite $\mathbb{F}_2$ class computation; the *reduction* of the categorical obstruction to that cochain complex is proved on paper and is not formalised. The state arithmetic (3)–(4) is likewise Lean-checked (`WorkersRetract.lean`, the retract $r \circ \sigma = \text{id}$). We flag the boundary rather than blur it. The general reduction of store-comonad interleaving to a distributive law is invoked here at the directed-container level – ΔS is a directed container – where it is available; a restatement directly for store comonads is not needed for this note and is recorded as an open problem in §7.

What threading does, and does not, do. Hand-threading two shared states does not escape the distributive law. Every call site that reads one register and writes another silently commits to an order of effects – an implicit λ – and nothing checks that the per-site choices are globally consistent, which is exactly what the coherence axioms demand. A program can hand-thread its way into a globally inconsistent interleaving one locally reasonable line at a time; that inconsistency is $[\omega] \neq 0$, discovered at run time as a re-entrancy bug. The store-comonad worker does not make the composite exist automatically – no formalism can, because sometimes it genuinely does not – but it *concentrates* the decision: the interleaving is

	single agent	interacting composite
store-comonad worker	$\text{Store}_S X = S \times X^S$	Zappa–Szép weld; needs a law λ
hand-threaded state	getter / setter	implicit λ per call site
what the comonad adds	<i>nothing</i> (a rename)	the interleaving as <i>one</i> named λ ; its validity as <i>one</i> class $[\omega]$, checkable before the system runs

TABLE 1. What the store-comonad reformulation buys, by system size.

one named object λ , its validity is one class $[\omega]$, and both can be inspected before the system runs, in one place, instead of being reconstructed from scattered call sites after it breaks. The ledger. Table 1 states the verdict as plainly as the question deserves. The reformulation buys nothing for a single agent, and it does not make composition automatic; what it buys is a change in *where* and *when* the one real decision of a composite – the interleaving of shared effects – is made and checked.

6. POSITIONING

The note sits one refinement past the two nearest semantic accounts of protocols, and beside a process-calculus line.

Session coalgebras. Keizer, Basold and Pérez [4] give a coalgebraic semantics of session types over exactly the container endofunctor $F(X) = \coprod_{a \in A} X^{B_a}$, which we adopt for the single step (Remark 1). Their object is a coalgebra $c : X \rightarrow F(X)$ for protocol *transitions* – a single one-step map, with state folded into the carrier X – and their composition is the π -calculus parallel product $P \mid Q$. They introduce no comonad, no cofree interaction trace, no substitution or tensor algebra of protocols, no store-comonad state, and no obstruction to composition. The comonadic / directed-container layer, the cofree trace, the $\triangleleft/\otimes$ algebra, the store worker, and the $[\omega]$ obstruction are outside their development, and are what this note contributes.

Effect handlers for prompt pipelines. Segura [7] models bounded LLM prompt pipelines with Lawvere theories and algebraic-effect handlers (retrieval, state, exceptions, typed tool invocation, with live/log/replay interpretations). This is the closest “algebra of tool-calling,” but it is protocol-agnostic: it axiomatises the effects a pipeline may perform, not the protocol object through which requests and responses flow, and it uses no polynomial-functor or container structure. We know this work at abstract level only (the full text is paywalled and we have not read it in depth); we position against it rather than build on it, and cite it as the effect-handler precedent that does not model the protocol object.

Process-calculus and other accounts. Schlapbach’s process-calculus treatment of protocol bisimulation is a natural target for a directed-container morphism reformulation, which we do not pursue here. A withdrawn attempt at a “monadic” context-engineering semantics (Zhang–Wang, arXiv:2512.22431, withdrawn for issues in its categorical foundations) indicates that the naive monadic route does not reach the structure; the comonadic route of this note is, we believe, the correct one, supplied for free by the equivalence chain.

7. CONCLUSION AND OPEN PROBLEMS

A tool-calling protocol is a container, its harness is that container’s comonad, its interaction trace is the cofree comonad, its stateful context is a store worker, and the correctness of a multi-agent interleaving is the vanishing of one cohomology class. None of these is a new theorem; together they say that the object applied category theory already studies – the directed container, equivalently the polynomial comonad – is the semantics of agent tool-calling, and that the multi-agent correctness question is not a testing problem but a computable obstruction. For an orchestration of many interacting agents, which is what an agentic system is, that obstruction is the compositional-correctness guarantee worth naming.

Three problems remain open.

- (1) *A store-comonad restatement of the obstruction.* Theorem 1 is invoked at the directed-container level, where ΔS lives. Restating the reduction “store-comonad interleaving $\Leftrightarrow$ distributive law $\Leftrightarrow [\omega]$ ” directly in the store-comonad vocabulary, and formalising the categorical-to-cochain reduction that Remark 3 leaves on paper, is a concrete next step.
- (2) *Beyond the smallest model.* The bit $[\omega] = \varepsilon \in \mathbb{Z}/2$ is the supervisor–worker case. The class for a weld with a larger freeness datum (three or more interacting agents, or non-abelian shared state) is the natural generalisation; the freeness condition (L) is what must first be understood in the many-agent case.
- (3) *Protocol morphisms.* Reformulating protocol refinement and bisimulation – including the process-calculus accounts – as directed-container morphisms would connect this semantics to the existing operational theory and is the route to comparing two harnesses for the same protocol.

Acknowledgements. For Robin and Neil. This note answers a question Robin posed about whether the store-comonad worker earns its keep, recast for an external audience; the internal house-doc version is companion material.

REFERENCES

- [1] D. Ahman and T. Uustalu. *Directed containers as categories*. Electron. Proc. Theor. Comput. Sci. (EPTCS) 207 (2016), 89–98. arXiv:1604.01187.
- [2] D. Ahman and T. Uustalu. *Distributive laws of directed containers*. Progress in Informatics 10 (2013), 3–18.
- [3] N. Gambino and J. Kock. *Polynomial functors and polynomial monads*. Math. Proc. Cambridge Philos. Soc. 154 (2013), no. 1, 153–192. arXiv:0906.4931.
- [4] A. C. Keizer, H. Basold, and J. A. Pérez. *Session coalgebras: a coalgebraic view on session types and communication protocols*. ACM Trans. Program. Lang. Syst. (TOPLAS), 2021. arXiv:2011.05712.
- [5] N. Niu and D. I. Spivak. *Polynomial functors: a mathematical theory of interaction*. arXiv:2312.00990, 2023.
- [6] D. I. Spivak. *A reference for categorical structures on Poly*. arXiv:2202.00534, 2022.
- [7] J. J. Segura. *Algebraic effects for bounded prompt pipelines: Lawvere theories, handlers, and outcome traces*. Computing 108 (2026). DOI 10.1007/s00607-026-01699-w. (Known to the author at abstract level only; full text paywalled.)
- [8] S. Katsumata, E. Rivas, and T. Uustalu. *Interaction laws of monads and comonads*. LICS 2020. arXiv:1912.13477.

KODAMAI